DOCUMENTO DE ARQUITECTURA DE SOLUCIONES (SAD)

SISTEMA DE BANCA POR INTERNET - BANCO BP

Devsu evaluación técnica

Autor: Fernando Gómez - Arquitecto de Soluciones Sénior
Estatus: Aprobado para Implementación
Tecnología Base: Amazon Web Services (AWS)
Metodología de Modelado: Modelo C4 (Niveles 1, 2 y 3)
Clasificación: Confidencial - Uso Exclusivo Banco BP

HISTORIAL DE CONTROL DE CAMBIOS

Versión	Fecha	Autor	Organización / Rol	Descripción del Cambio
v1.0	11-Oct-2026	Arquitecto de Soluciones	Banco BP / Consultor	**Versión Final Consolidada.** Inclusión de Infraestructura AWS, sección Normativa y Glosario. Listo para Defensa Técnica.

Devsu evaluación técnica

Tabla de Contenidos

Nivel 1: Diagrama de Contexto del Sistema (Modelo C4)
- 1.1. Descripción del Sistema Central
- 1.2. Actores (Personas)
- 1.3. Sistemas Externos y Dependencias (Ecosistema BP e Interbancario)
- 1.4. Interacciones y Flujos de Información a Alto Nivel
- 1.5. Estrategia Inicial para Atributos de Calidad (Nube AWS)

Nivel 2: Diagrama de Contenedores (Modelo C4)
- 2.1. Selección y Justificación del Front-End
- 2.1.1. Aplicación Web: React.js
- 2.1.2. Aplicación Móvil: Dos Opciones y Justificación Técnica
- 2.2. Diseño de la Capa de Integración y Microservicios
- 2.2.1. Puerta de Enlace: Amazon API Gateway
- 2.2.2. Servicios Propuestos (Capa de Microservicios)
- 2.3. Diagrama de Contenedores
- 3. Arquitectura de Autenticación, Seguridad y Onboarding
- 3.1. Recomendación del Flujo de Autenticación (Estándar OAuth 2.0)
- 3.2. Flujo de Onboarding Digital con Reconocimiento Facial
- 3.3. Métodos de Acceso Posteriores y Robustecimiento Seguridad

Nivel 3: Diseño de Componentes (Modelo C4) y Patrones
- 4.1. Patrón de Diseño para Clientes Frecuentes: Cache-Aside y Proxy
- 4.2. Estrategia para la Base de Datos de Auditoría e Inmutabilidad
- 4.3. Estructura Interna de Componentes (Nivel 3 C4)
- 4.4. Optimización de Rendimiento en Llamadas Concurrentes

Consolidación de Infraestructura y Resiliencia en AWS
- 5.1. Alta Disponibilidad (HA) y Excelencia Operativa
- 5.2. Tolerancia a Fallos, Auto-healing y Recuperación ante Desastres (DR)
- 5.3. Seguridad Concéntrica (Zero-Trust Network)
- 5.4. Monitoreo Avanzado y Observabilidad
- 5.5. Cumplimiento Normativo y Regulación Financiera

ANEXO: GLOSARIO DE TÉRMINOS Y ACRÓNIMOS

Documento técnica

Nivel 1: Diagrama de Contexto del Sistema (Modelo C4)

1.1. Descripción del Sistema Central

El **Sistema de Banca por Internet de BP** es la plataforma centralizada que permitirá a los clientes financieros gestionar sus finanzas de forma digital. El sistema actúa como un orquestador seguro y de alta disponibilidad que consolida la información financiera del cliente, facilita transacciones monetarias internas e interbancarias, y asegura el cumplimiento normativo mediante auditoría y notificaciones proactivas.

1.2. Actores (Personas)

- **Cliente Existente:** Usuario registrado que accede a la plataforma a través de canales digitales para consultar movimientos, saldos, realizar transferencias y pagos. Autenticado mediante credenciales tradicionales o biometría.
- **Nuevo Cliente (En Onboarding):** Usuario no registrado que interactúa con la aplicación móvil para dar de alta su cuenta mediante un proceso digitalizado que incluye validación de identidad por reconocimiento facial.

1.3. Sistemas Externos y Dependencias (Ecosistema BP e Interbancario)

El sistema no almacena de forma primaria los balances de las cuentas ni las identidades base, sino que interactúa con los siguientes sistemas del banco y de terceros:

Sistemas Internos de BP

- **Plataforma Core Bancario:** Sistema transaccional central de BP. Provee la información básica de cuentas, saldos, productos financieros e histórico de movimientos estándar.
- **Sistema de Información Detallada:** Base de datos o servicio complementario que almacena datos profundos del cliente (direcciones históricas, perfiles de riesgo, detalles extendidos de contacto) requeridos para transacciones complejas o auditoría.
- **Servicio de Identidad Corporativo (IdP):** Producto existente en BP compatible con **OAuth 2.0**. Se encarga de la emisión de tokens, validación de credenciales y centralización de identidades.

Revisión evaluación técnica

Sistemas Externos (Terceros y Proveedores)

- **Plataforma de Notificaciones 1 (SMS Gateway):** Servicio externo (ej. *AWS Endpoints / Twilio*) utilizado para el envío inmediato de códigos OTP (One-Time Password) y alertas transaccionales críticas.
- **Plataforma de Notificaciones 2 (Email/Push Service):** Sistema secundario o propio (ej. *Amazon SES* para correos y *Amazon SNS* para notificaciones push en móviles) encargado de enviar comprobantes de transferencias y estados de cuenta.
- **Red de Compensación Interbancaria (ACH / SWIFT / Cámara Local):** Entidad financiera externa que procesa, liquida y compensa las transferencias monetarias dirigidas hacia otros bancos.
- **Motor de Biometría y Reconocimiento Facial:** Servicio especializado de procesamiento de imágenes (propuesto: *Amazon Rekognition*) utilizado durante el Onboarding para validar la prueba de vida y contrastar la identidad con el documento legal del cliente.

1.4. Interacciones y Flujos de Información a Alto Nivel

Para visualizar el Contexto de C4, las relaciones se definen bajo el siguiente flujo lógico:

Origen	Destino	Protocolo / Mecanismo	Descripción
Cliente Existente	Sistema Banca BP	HTTPS	Consulta saldos, realiza transferencias y gestiona sus productos.
Nuevo Cliente	Sistema Banca BP	HTTPS	Inicia flujo de Onboarding capturando su rostro y datos personales.
Sistema Banca BP	Servicio de Identidad BP	OAuth 2.0 / OIDC	Delega la autenticación y solicita la validación de tokens de acceso.
Sistema Banca BP	Plataforma Core Bancario	REST / gRPC	Solicita datos básicos, saldos y ejecuta transferencias entre cuentas propias.
Sistema Banca BP	Sistema Info. Detallada	REST / SOAP	Consume datos extendidos del cliente para validaciones de seguridad.

Devsu evaluación técnica

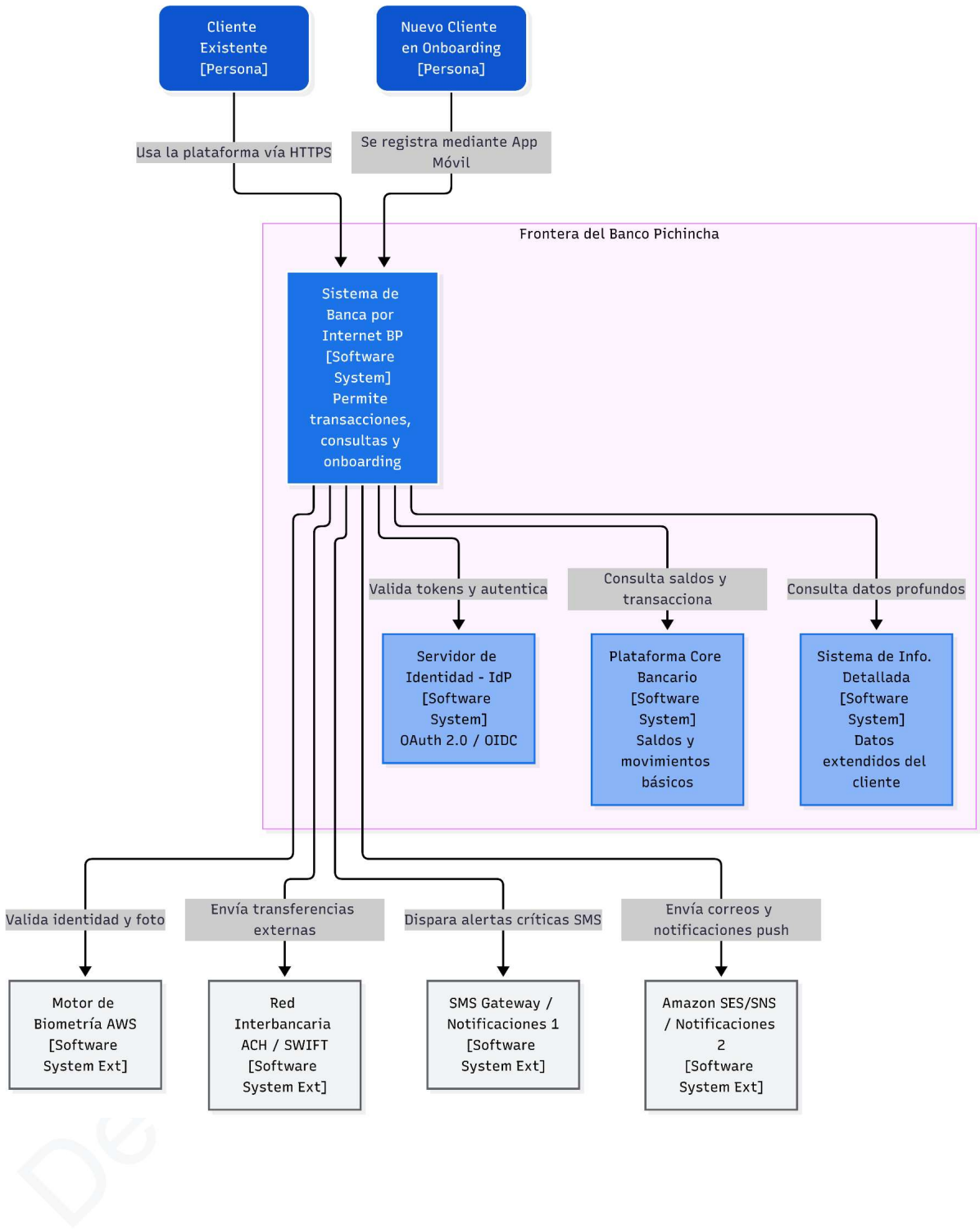
Sistema Banca BP	Motor de Biometría (AWS)	SDK / API HTTPS	Envía la captura facial para validación de identidad e inscripción inicial.
Sistema Banca BP	Red Interbancaria	ISO 20022 / API	Envía órdenes de pago para cuentas en otras entidades financieras.
Sistema Banca BP	Notificaciones 1 & 2	API REST / SMTP	Dispara alertas automáticas (SMS/Email/Push) ante cualquier movimiento.

1.5. Estrategia Inicial para Atributos de Calidad (Nube AWS)

Para cumplir con las restricciones del negocio, el Nivel 1 adopta las siguientes estrategias que gobernarán los niveles inferiores:

- **Baja Latencia y HA:** Despliegue en **Multi-AZ** (Múltiples Zonas de Disponibilidad) utilizando balanceadores de carga (*AWS ALB*) y redes de entrega de contenido (*Amazon CloudFront*) para optimizar el tráfico de las aplicaciones Front.
- **Seguridad:** Todo el tráfico viaja encriptado en tránsito (TLS 1.3) y en reposo (AES-256 mediante AWS KMS).

Devsu evaluación técnica


Cliente Existente [Persona]
Nuevo Cliente en Onboarding [Persona]
Usa la plataforma vía HTTPS
Se registra mediante App Móvil
Frontera del Banco Pichincha
Sistema de Banca por Internet BP [Software System] Permite transacciones, consultas y onboarding
Valida tokens y autentica
Consulta saldos y transacciona
Consulta datos profundos
Servidor de Identidad - IdP [Software System] OAuth 2.0 / OIDC
Plataforma Core Bancario [Software System] Saldos y movimientos básicos
Sistema de Info. Detallada [Software System] Datos extendidos del cliente
Valida identidad y foto
Envía transferencias externas
Dispara alertas críticas SMS
Envía correos y notificaciones push
Motor de Biometría AWS [Software System Ext]
Red Interbancaria ACH / SWIFT [Software System Ext]
SMS Gateway / Notificaciones 1 [Software System Ext]
Amazon SES/SNS / Notificaciones 2 [Software System Ext]

Nivel 2: Diagrama de Contenedores (Modelo C4)

2.1. Selección y Justificación del Front-End

El requerimiento exige una aplicación web tipo **SPA (Single Page Application)** y una **App Móvil** desarrollada sobre un framework multiplataforma. Como Arquitecto de Soluciones, se plantean las siguientes definiciones tecnológicas:

2.1.1. Aplicación Web: React.js

Para la SPA de banca por internet se selecciona **React.js**.

- **Justificación:** Cuenta con el ecosistema de componentes reutilizables más maduro del mercado. Facilita la creación de arquitecturas basadas en microfrontends (vía Module Federation) si el banco decide escalar canales independientes en el futuro. Su renderizado eficiente y soporte nativo para TypeScript garantizan una experiencia segura, óptima y con una curva de mantenimiento predecible.

2.1.2. Aplicación Móvil: Dos Opciones y Justificación Técnica

Criterio Técnico	Opción 1: Flutter (Recomendada)	Opción 2: React Native
Arquitectura de Renderizado	**Motor Gráfico Skia/Impeller nativo.** Dibuja directamente en el lienzo del dispositivo sin pasar por un puente JavaScript.	**Puente Nativo / JSI.** Traduce componentes de JavaScript a componentes nativos del sistema operativo (iOS/Android).
Rendimiento e Interfaz	**Rendimiento superior (60-120 FPS fijos).** Garantiza animaciones ultra fluidas y transiciones instantáneas óptimas para el usuario financiero.	**Rendimiento alto,** pero propenso a micro-stuttering si se satura el puente nativo con cálculos complejos o manejo pesado de datos.
Seguridad y Criptografía	**Compilación nativa directa a código máquina (AOT - Ahead of Time).** Hace que la ingeniería inversa del binario (*reverse engineering*) sea extremadamente difícil.	**Binario empaquetado con Bundle de JavaScript.** Aunque se use obfuscación, el código fuente es más susceptible a descompilación.

Devuelta Evaluación Técnica

Manejo de Biometría	Integración nativa directa y limpia a través de plugins maduros con las APIs de *FaceID / BiometricPrompt*.	Integración mediante librerías de la comunidad que dependen del ciclo de vida del puente reactivo.

Justificación Final de la Elección: Se recomienda **Flutter** para el canal móvil de BP. En una aplicación bancaria de misión crítica, la **seguridad del binario compilado** y la **fluidez de la interfaz gráfica** (crucial para flujos delicados como capturas faciales de Onboarding) son prioridades absolutas. Flutter mitiga riesgos de rendimiento asociados al procesamiento de imágenes pesadas en tiempo real.

2.2. Diseño de la Capa de Integración y Microservicios

Para lograr baja latencia, aislamiento térmico de fallos y una arquitectura desacoplada, el backend se estructurará bajo un enfoque basado en capacidades de negocio y microservicios independientes en **AWS**.

2.2.1. Puerta de Enlace: Amazon API Gateway

Actúa como el único punto de entrada para los canales digitales (Web y Móvil). Sus funciones principales incluyen:

- **Seguridad:** Enrutamiento SSL/TLS termination, integración con el IdP corporativo para la validación del JSON Web Token (JWT), control de cuotas (*Rate Limiting*) y protección contra ataques volumétricos (*DDoS*) vía AWS Shield.
- **Desacoplamiento:** Redirige las peticiones HTTPS entrantes a los microservicios internos mediante rutas REST normalizadas.

2.2.2. Servicios Propuestos (Capa de Microservicios)

Para maximizar el rendimiento y la cohesión de la información para el cliente, expandimos los 3 servicios iniciales a un catálogo optimizado de **5 microservicios**:

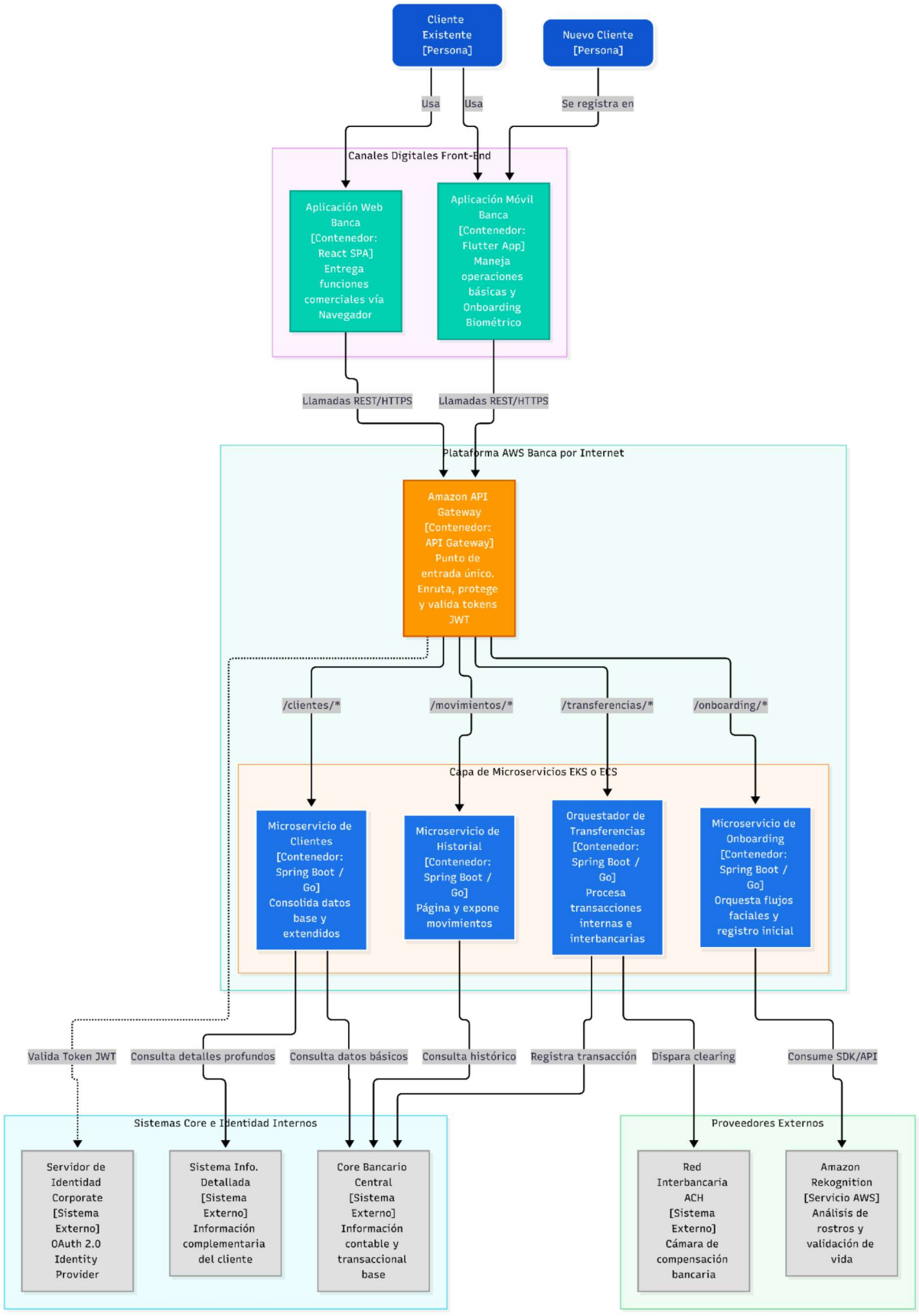
1. **Servicio de Onboarding y Biometría:** Orquesta el flujo de registro de nuevos clientes. Coordina la carga temporal de la selfie/documento y gestiona la llamada al motor de validación facial (*Amazon Rekognition*).
2. **Servicio de Datos Básicos de Clientes:** Expone el perfil del usuario. Integra y consolida en paralelo la información básica del *Core Bancario* con los datos extendidos del *Sistema de Información Detallada*.
3. **Servicio de Posición Consolidada y Productos:** Encargado exclusivo de listar saldos de cuentas corrientes, ahorros y tarjetas. Consume directamente del *Core*.

Devsu evaluación técnica

4. **Servicio de Movimientos e Historial:** Expone el histórico transaccional. Implementa estrategias de paginación para mitigar la latencia de las consultas masivas hacia el Core.
5. **Servicio de Transferencias y Pagos (Orquestador Transaccional):** Maneja transacciones entre cuentas propias y remite las transacciones hacia entidades externas utilizando un patrón asíncrono para transferencias interbancarias (*Red ACH*), previniendo tiempos de espera prolongados en el Front-end.

Devsu evaluación técnica

2.3. Diagrama de Contenedores



3. Arquitectura de Autenticación, Seguridad y Onboarding

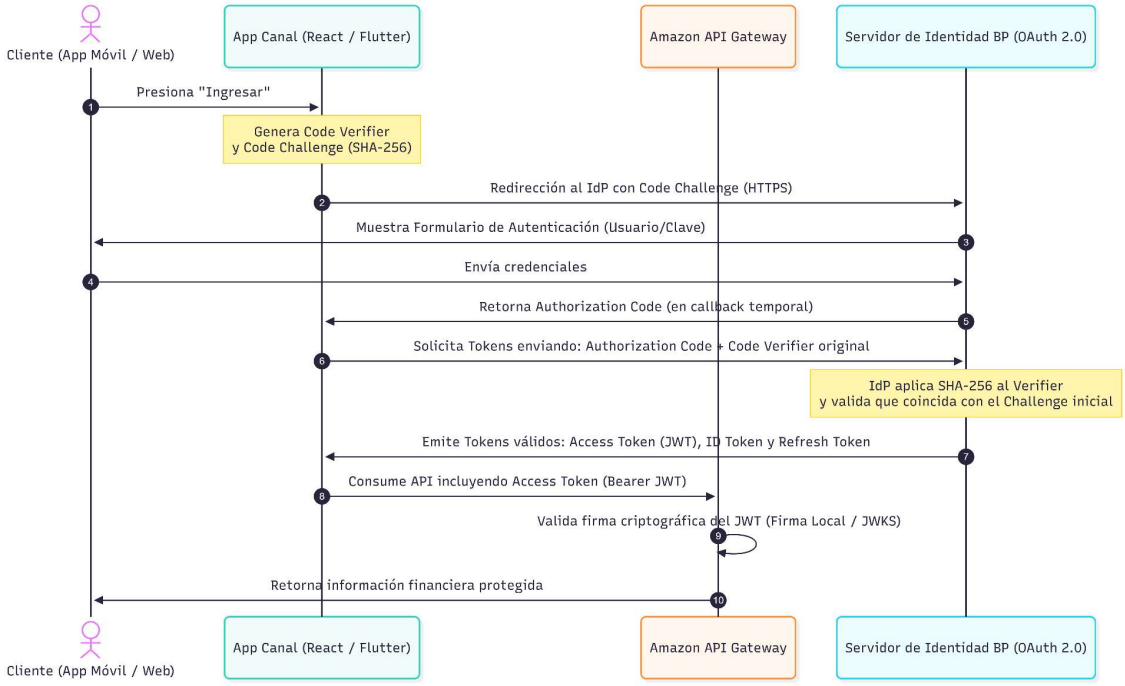
3.1. Recomendación del Flujo de Autenticación (Estándar OAuth 2.0)

Para proteger los canales digitales de BP, se prohíbe el uso de flujos antiguos como *Implicit Grant* (inseguro para SPA) o *Resource Owner Password Credentials Grant* (expone las credenciales directamente a la aplicación).

El Flujo Recomendado: Authorization Code Grant con PKCE (Proof Key for Code Exchange)

Tanto para la aplicación web (**React SPA**) como para la aplicación móvil (**Flutter App**), el estándar actual de la industria (según la especificación **RFC 7636**) exige implementar **Authorization Code con PKCE**.

- **¿Por qué PKCE?** Elimina la necesidad de almacenar un *Client Secret* (secreto del cliente) dentro de las aplicaciones Front-end (donde un atacante podría descompilar el binario de Flutter o inspeccionar el código de React).
- **Mecanismo de mitigación:** Reemplaza el secreto estático por un secreto dinámico generado por sesión: un verificador de código aleatorio (*Code Verifier*) y su respectiva firma hash (*Code Challenge*). Esto previene ataques de interceptación de código de autorización en el dispositivo o navegador.

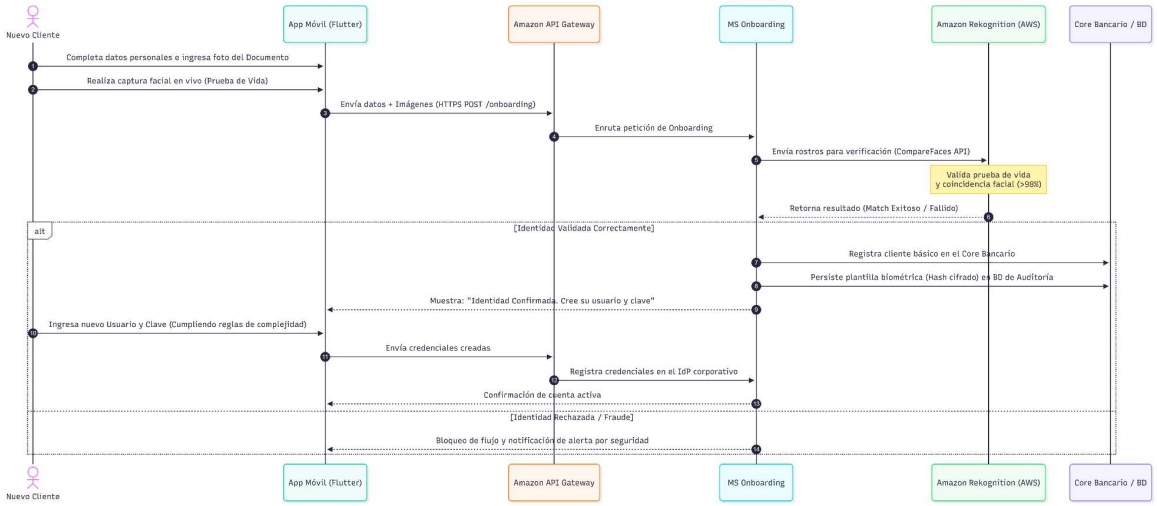


3.2. Flujo de Onboarding Digital con Reconocimiento Facial

Para los nuevos clientes de la aplicación móvil, se diseña un flujo de **Onboarding Desacoplado** que valida la identidad mediante biometría antes de crear las credenciales permanentes del sistema.

Componentes Tecnológicos Propuestos:

1. **Captura en el Front (Flutter):** Se utiliza la cámara del dispositivo móvil para forzar una captura guiada (detección de parpadeo, giro de cabeza) para asegurar **Prueba de Vida (Liveness Detection)** pasiva o activa.
2. **Procesamiento en AWS (Amazon Rekognition):** El microservicio de Onboarding recibe la imagen, invoca las APIs de Rekognition para comparar el rostro capturado en vivo contra la fotografía del documento de identidad legal cargado (cédula/pasaporte), devolviendo un porcentaje de similitud (*Confidence Score*).



3.3. Métodos de Acceso Posteriores y Robustecimiento Seguridad

Una vez que el usuario finaliza con éxito su proceso de Onboarding, la arquitectura de la aplicación móvil debe registrar mecanismos de autenticación rápidos y avanzados para evitar tener que solicitar la clave alfanumérica en cada sesión.

- **Autenticación Biométrica Nativa (Huella / Rostro del OS):** La aplicación de **Flutter** interactuará directamente con el hardware del teléfono mediante las APIs nativas del sistema operativo (*BiometricPrompt* en Android y *LocalAuthentication* en iOS / FaceID / TouchID).
    1. **Mecanismo de Seguridad:** La aplicación **nunca** guarda la huella digital en los servidores del banco ni en la base de datos (por regulaciones de privacidad estricta). El teléfono valida la biometría

internamente en su chip seguro (*Secure Enclave / Trusted Execution Environment*) y, al ser exitosa, descifra un token de sesión seguro almacenado en el llavero de almacenamiento seguro local (*iOS Keychain / Android Keystore*).

- **Herramientas para Robustecer la Aplicación:**
  1. **AWS Cognito o Ping Identity:** Si el producto de identidad nativo del banco requiere un complemento para autenticación multifactor (MFA), estas herramientas pueden actuar como brokers federados.
  2. **Twilio Verify / AWS SNS:** Para el envío automatizado de contraseñas de un solo uso (OTP) a través de SMS y notificaciones Push firmadas para autorizar transferencias monetarias de alto valor.
  3. **WAF (Web Application Firewall):** Acoplado al API Gateway para bloquear de forma proactiva inyecciones SQL, Cross-Site Scripting (XSS) y ataques automatizados de bots que intenten adivinar credenciales bancarias.

Nivel 3: Diseño de Componentes (Modelo C4) y Patrones

Para este nivel, nos enfocaremos en la estructura interna de dos de los contenedores más críticos del sistema distribuidos en la nube de AWS: el **Orquestador de Transferencias** y el **Microservicio de Clientes (Datos Básicos)**.

4.1. Patrón de Diseño para Clientes Frecuentes: Cache-Aside y Proxy

El requerimiento exige un mecanismo de persistencia optimizado para clientes frecuentes. Para evitar saturar el *Core Bancario* y el *Sistema de Información Detallada* con consultas repetitivas de usuarios de alta transaccionalidad, implementamos una combinación de patrones:

- **Patrón Proxy / Decorador:** El microservicio de clientes expone una interfaz de consulta de datos. Una clase Proxy intercepta las solicitudes de lectura. Si los datos del cliente ya están en memoria rápida, los retorna inmediatamente; si no, delega la llamada a la capa de integración de los Cores.
- **Patrón Cache-Aside (Lectura/Escritura en Caché):** Implementado utilizando **Amazon ElastiCache (Redis)**.
  - **Lógica de Lectura:** Cuando ingresa una petición, se verifica la existencia del perfil en el cluster de Redis mediante una clave única (`cliente:frecuente:{id}`). Si existe (*Cache Hit*), se devuelve en milisegundos de forma local. Si no existe (*Cache Miss*), se consulta a

Documentación técnica

los Cores, se almacena el resultado en Redis con un tiempo de vida corto (TTL de 15 a 30 minutos) y se retorna al usuario.
    - **Evicción Automática:** Redis gestionará de forma transparente la memoria mediante políticas LRU (Least Recently Used), manteniendo almacenados únicamente a los clientes verdaderamente frecuentes y liberando espacio de forma autónoma (Auto-healing de memoria).

4.2. Estrategia para la Base de Datos de Auditoría e Inmutabilidad

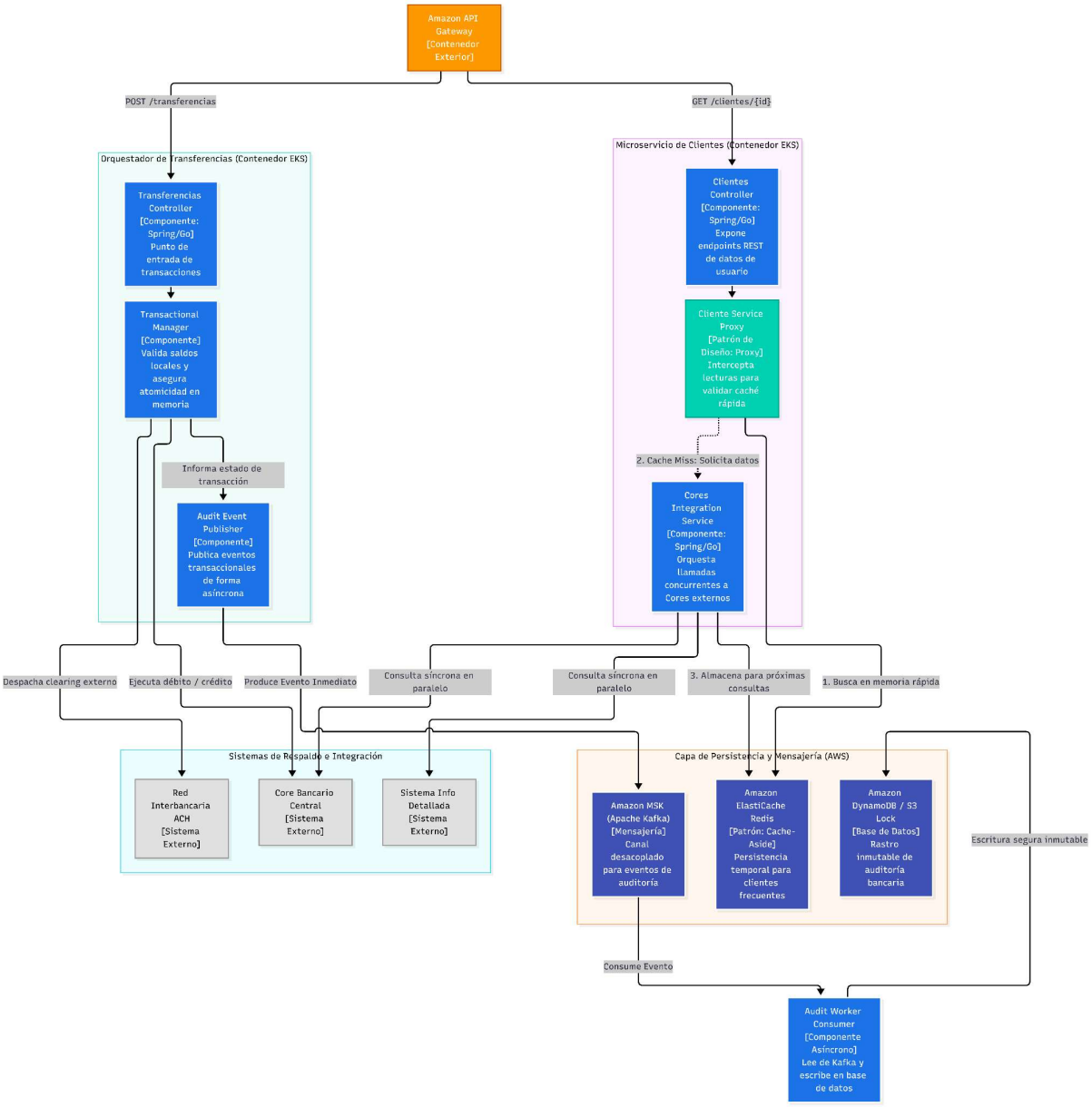
Registrar de manera segura todas las acciones del cliente requiere un mecanismo tolerante a fallos que no afecte el rendimiento de las operaciones en vivo.

- **Arquitectura Orientada a Eventos (EDA):** Los microservicios no escriben directamente en la base de datos de auditoría de forma síncrona. En su lugar, cada acción (ej. `TRANSFERENCIA_CREADA`, `LOGIN_EXITOSO`) publica un evento en un tópico de alta velocidad de **Amazon MSK (Managed Streaming for Apache Kafka)**.
- **Desacoplamiento y Persistencia:** Un componente consumidor asíncrono (*Audit Consumer*) lee de forma dedicada los mensajes de Kafka y los escribe en la base de datos de auditoría.
- **Tecnología Propuesta para Auditoría: Amazon DynamoDB + Amazon Timestream**
    - **DynamoDB:** Almacena el rastro de auditoría de estado actual (*Audit Trail*) debido a su capacidad ilimitada de escalabilidad horizontal bajo demanda.
    - **Amazon QLDB o S3 con Object Lock (Cumplimiento de Norma):** Para garantizar la inmutabilidad legal absoluta (que nadie, ni un administrador de base de datos, pueda alterar los registros), los logs de auditoría se archivan en un bucket de **Amazon S3** con la política *Object Lock* activa (WORM: Write Once, Read Many).

Documentación Técnica

4.3. Estructura Interna de Componentes (Nivel 3 C4)

Arquitectura interna y la interacción de patrones dentro de la capa de microservicios:



4.4. Optimización de Rendimiento en Llamadas Concurrentes

Para evitar que el *Microservicio de Clientes* sufra degradación de latencia al consumir de dos plataformas independientes (Core Bancario e Información Detallada), el `Cores Integration Service` implementa programación reactiva o asíncrona no bloqueante:

- Al recibir la solicitud, el componente dispara dos hilos de consulta independientes en paralelo (*Futures/Promises* o *Coroutines*) utilizando un pool de conexiones optimizado.

- Aplica el patrón **Circuit Breaker** (vía *Resilience4j*). Si el *Sistema de Información Detallada* presenta lentitud o caídas, el circuito se abre y el sistema responde inmediatamente degradando la interfaz de forma segura (retornando solo los datos del Core y un aviso temporal), garantizando **tolerancia a fallos** sin colgar las peticiones del usuario.

Consolidación de Infraestructura y Resiliencia en AWS

Con la lógica de componentes resuelta, diseñaremos el despliegue físico y de red en la infraestructura de nube para garantizar los atributos de misión crítica solicitados.

5.1. Alta Disponibilidad (HA) y Excelencia Operativa

- **Despliegue Multi-AZ:** Toda la capa de cómputo se ejecutará sobre un clúster de **Amazon EKS (Elastic Kubernetes Service)** distribuido de forma redundante en un mínimo de 3 Zonas de Disponibilidad (AZ) independientes.
- **Capacidad Auto-scaling:** Se implementará *Karpenter* o *Cluster Autoscaler* en EKS para escalar los nodos físicos dinámicamente según la carga de CPU/Memoria, asegurando baja latencia en picos transaccionales.
- **Capa de Datos Resiliente:** *Amazon ElastiCache (Redis)* se configurará en modo de replicación Multi-AZ con failover automático.

5.2. Tolerancia a Fallos, Auto-healing y Recuperación ante Desastres (DR)

- **Auto-healing de Aplicación:** EKS validará la salud de los contenedores mediante *Liveness* y *Readiness Probes*. Si un microservicio se corrompe o satura, Kubernetes destruirá el pod de forma inmediata y levantará uno nuevo sano automáticamente sin afectar al usuario final.
- **Estrategia de DR (Disaster Recovery): Active-Passive (Warm Standby) o Active-Active:**
    - Dado que contamos con presupuesto suficiente y es una aplicación bancaria, adoptaremos un enfoque de **Multi-Región Warm Standby**. La región primaria (ej. *us-east-1*) procesará el 100% del tráfico. Una región secundaria (ej. *us-west-2*) mantendrá la infraestructura replicada y lista para encenderse si ocurre una catástrofe total en la costa este. El tráfico se redirigirá en minutos mediante políticas de salud en **Amazon Route 53**.

Documento de evaluación técnica

5.3. Seguridad Concéntrica (Zero-Trust Network)

- **Aislamiento de Red (VPC):** La infraestructura se dividirá en 3 capas de subredes estrictas:
    1. *Subred Pública:* Hospeda únicamente el *Application Load Balancer (ALB)* y los endpoints de *CloudFront*.
    2. *Subred Privada (Cómputo):* Donde residen los nodos de EKS con los microservicios. No tienen acceso directo desde internet.
    3. *Subred Aislada (Datos):* Los clústeres de Redis, Kafka y las llaves de encriptación. Solo aceptan tráfico originado dentro de la subred privada de cómputo.
- **Protección Perimetral:** Implementación de **AWS WAF** integrado a API Gateway para mitigar los OWASP Top 10 y **AWS Shield Advanced** para neutralizar ataques de denegación de servicio (DDoS).

5.4. Monitoreo Avanzado y Observabilidad

- **Amazon CloudWatch & AWS X-Ray:** Centralización de logs de aplicación y métricas de infraestructura. X-Ray inyectará un ID de correlación en las cabeceras HTTP desde la SPA/Móvil para realizar un rastreo distribuido (*Distributed Tracing*) de punta a punta, permitiendo identificar exactamente qué microservicio o sistema externo (Core) causa latencia en una transacción específica.

5.5. Cumplimiento Normativo y Regulación Financiera

Para operar de manera legal y mitigar riesgos sancionatorios, la arquitectura del sistema de Banca por Internet de BP implementa controles tecnológicos específicos que responden a las siguientes normativas:

1. Ley Orgánica de Protección de Datos Personales (LOPDP - Ecuador) / GDPR

Dado que el sistema manipula información sensible de clientes (datos básicos, perfiles financieros y biometría facial), se establecen los siguientes mecanismos:

- **Principio de Minimización y Aislamiento:** Los datos biométricos recolectados durante el onboarding se transforman de inmediato en un hash matemático irreversible (plantilla vectorizada). Las imágenes originales (*selfies*) se eliminan de las zonas de tránsito temporal tras recibir el *score* de validación de *Amazon Rekognition*.
- **Cifrado de Extremo a Extremo:** Toda la información en reposo dentro de *Amazon DynamoDB* y los cachés de *Amazon ElastiCache (Redis)* se encripta

Devs: evaluación técnica

utilizando llaves custodiadas en **AWS KMS (Key Management Service)** con rotación automática anual. El acceso de lectura a estas llaves está restringido estrictas políticas IAM bajo el principio de menor privilegio.

2. Norma de Control para la Gestión de Riesgos de Tecnología de la Información (Superintendencia de Bancos)

Esta regulación exige una estricta trazabilidad de las operaciones y resiliencia ante desastres:

- **Inmutabilidad y No Repudio (Auditoría):** La solución basada en *Amazon MSK (Kafka)* que envía logs transaccionales a un bucket de *Amazon S3* configurado con **Object Lock (WORM: Write Once, Read Many)** garantiza que ninguna acción del cliente o administrador pueda ser alterada, borrada o falsificada. Esto cumple con los plazos legales de retención de registros históricos exigidos por el ente de control.
- **Tolerancia y Continuidad del Negocio:** La estrategia de **Disaster Recovery Multi-Región (Warm Standby)** diseñada en el punto 5.2 responde directamente a los tiempos de recuperación máximos permitidos (RTO y RPO) exigidos por la Superintendencia para canales electrónicos de atención masiva.

3. Estándar de Seguridad de Datos de la Industria de Tarjetas de Pago (PCI-DSS)

Aunque el sistema gestiona movimientos y transferencias de cuentas, para la visualización o futuros pagos con tarjetas de débito/crédito asociadas a los productos del cliente, la arquitectura define:

- **Tokenización de Datos de Tarjetas:** El API Gateway y los microservicios transaccionales nunca almacenan ni exponen el número de tarjeta completo (PAN). Se implementa un servicio de tokenización periférico para enmascarar los datos sensibles en los canales digitales (React y Flutter).

4. Estándar ISO 20022 (Mensajería Financiera)

- **Capa de Integración de Pagos:** El *Orquestador de Transferencias* utiliza estructuras de mensajería basadas en XML/JSON que siguen el estándar internacional ISO 20022 para la comunicación con la *Red de Compensación Interbancaria (ACH)*, facilitando el procesamiento seguro y estandarizado de pagos interbancarios.

ANEXO: GLOSARIO DE TÉRMINOS Y ACRÓNIMOS

- **ACH (Automated Clearing House):** Red centralizada de compensación electrónica que procesa y liquida transferencias financieras interbancarias de forma diferida.
- **API Gateway:** Componente perimetral que actúa como punto de entrada único para gestionar, enrutar, autenticar y aplicar políticas de control de tráfico (*Rate Limiting*) sobre las llamadas hacia los microservicios.
- **AWS KMS (Key Management Service):** Servicio administrado en la nube para la creación, custodia y rotación criptográfica de llaves criptográficas utilizadas para cifrar datos en reposo.
- **Cache-Aside:** Patrón de diseño de software donde la aplicación consulta primero la memoria caché; si los datos no están (*Cache Miss*), los lee de la base de datos origen y los escribe en la caché para futuras consultas.
- **Circuit Breaker (Disyuntor):** Patrón de diseño de resiliencia que detecta fallas o degradaciones de latencia en servicios externos y abre el circuito para rechazar peticiones de inmediato, previniendo caídas en cascada del sistema.
- **C4 Model:** Metodología de documentación de arquitectura de software que desglosa los sistemas en cuatro niveles de abrazamiento visual progresiva: Contexto, Contenedores, Componentes y Código.
- **Distributed Tracing (Rastreo Distribuido):** Técnica de monitoreo que inyecta un ID único en el flujo de una petición para medir el tiempo exacto de ejecución en cada microservicio e infraestructura que atraviesa.
- **IdP (Identity Provider):** Sitema central encargado de autenticar identidades de usuarios, validar credenciales y emitir tokens de seguridad bajo estándares federados.
- **ISO 20022:** Estándar internacional para el intercambio de mensajes financieros electrónicos entre entidades bancarias de todo el mundo.
- **Liveness Detection (Prueba de Vida):** Tecnología biométrica que valida si la muestra facial capturada por una cámara proviene de un ser humano vivo presente en tiempo real, mitigando fraudes por fotos o videos fijos.
- **LOPDP:** Ley Orgánica de Protección de Datos Personales de Ecuador, que regula las obligaciones de las instituciones sobre el tratamiento, almacenamiento y privacidad de los datos de los ciudadanos.
- **Multi-AZ (Multi-Availability Zone):** Estrategia de despliegue en la nube donde los sistemas se replican en múltiples centros de datos físicamente independientes dentro de una misma región geográfica para asegurar alta disponibilidad.
- **Object Lock (WORM):** Característica de almacenamiento (*Write Once, Read Many*) que bloquea criptográficamente los archivos para impedir que cualquier usuario pueda borrarlos, mutarlos o alterarlos durante un tiempo definido.

- **PKCE (Proof Key for Code Exchange):** Extensión del protocolo OAuth 2.0 (RFC 7636) diseñada para asegurar flujos de autenticación en aplicaciones públicas (móviles y SPA) eliminando la necesidad de almacenar contraseñas secretas estáticas.
- **SPA (Single Page Application):** Aplicación web que se ejecuta en el navegador del usuario y actualiza dinámicamente el contenido sin recargar la página completa, consumiendo datos a través de APIs de backend.
- **Warm Standby:** Estrategia de recuperación ante desastres donde una infraestructura secundaria en otra región se mantiene encendida en su escala mínima con datos replicados, lista para asumir el tráfico operativo en minutos tras un fallo general.

Devsu evaluación técnica